

O&G · 2 — Neural ODEs for production that doesn't follow Arps ⇨

Arps decline curves ([01_decline_curve_analysis.jl](#)) assume smooth, boundary-dominated depletion. Real wells often deviate: workovers, temporary shut-ins, choke changes, and gauge-cycling all leave signatures a simple decline law can't represent. A **neural ODE** $\dot{q} = f_{\theta}(q, t)$ makes no such assumption — it learns whatever dynamics the data actually show.

```
1 using Lux, OrdinaryDiffEq, SciMLSensitivity, Optimization, OptimizationOptimisers,  
   ComponentArrays, Random, Plots
```

1. A well with an irregular production history ⇨

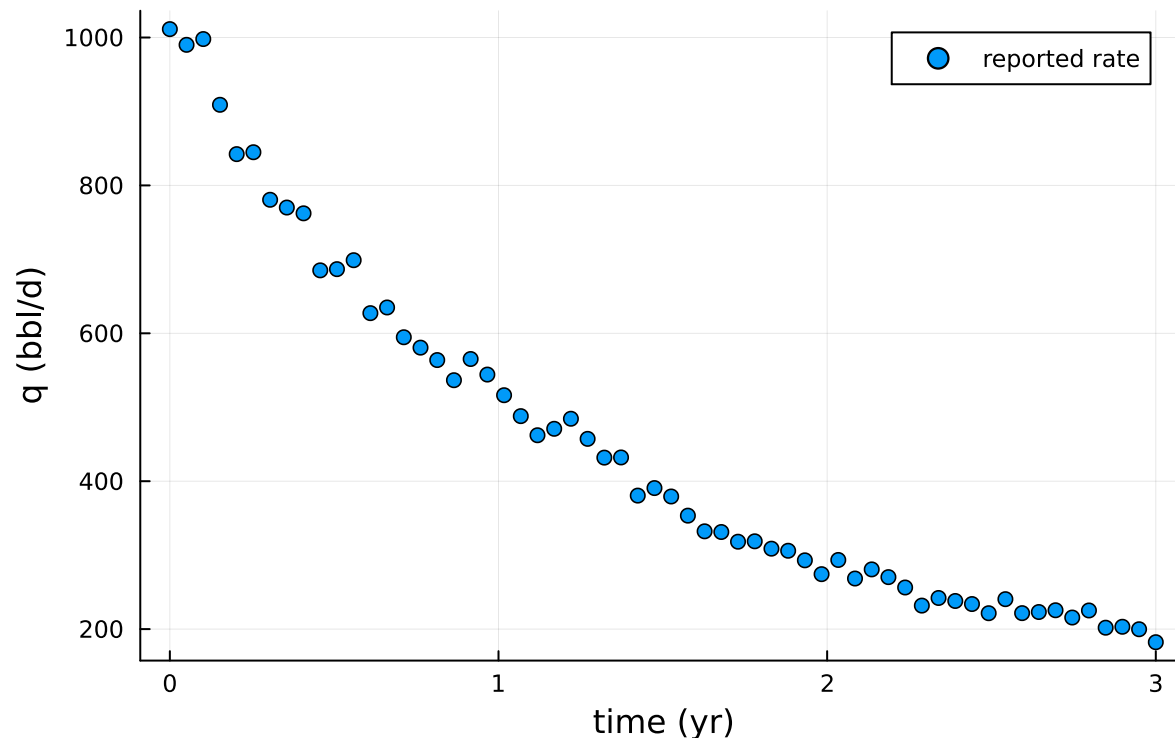
We generate synthetic data from an *underlying* hyperbolic decline that is disturbed by a temporary shut-in (a dip around 1.5 years, e.g. for a workover) and a small periodic ripple (choke cycling). In a real dataset we would not know this structure — we only see the noisy rate history.

```
true_well_dynamics! (generic function with 1 method)  
1 function true_well_dynamics!(du, u, p, t)  
2     q = max(u[1], 0.0) # production rate can't go negative; guards the fractional  
   power below  
3     qi, Di, b = p  
4     arps_term = -Di * q^(1 + b) / qi^b  
5     workover_dip = -0.6 * q * exp(-((t - 1.5)^2) / (2 * 0.06^2))  
6     choke_ripple = 0.08 * q * sin(2π * t / 0.25)  
7     du[1] = arps_term + workover_dip + choke_ripple  
8     return nothing  
9 end
```

```
1x60 Matrix{Float64}:
1011.35  990.104  997.922  909.012  842.312  ...  201.878  203.149  199.807  182.288
```

```
1 begin
2   rng_og2 = Random.default_rng()
3   Random.seed!(rng_og2, 5)
4
5   true_p_og2 = (1000.0, 0.8, 0.5) # qi, Di, b
6   u0_og2 = [1000.0]
7   tspan_og2 = (0.0, 3.0)
8   tsteps_og2 = range(tspan_og2[1], tspan_og2[2]; length=60)
9
10  true_prob_og2 = ODEProblem(true_well_dynamics!, u0_og2, tspan_og2, true_p_og2)
11  true_sol_og2 = solve(true_prob_og2, Tsit5(); saveat=tsteps_og2)
12  well_data = Array(true_sol_og2) .* (1 .+ 0.03 .* randn(rng_og2, 1,
13    length(tsteps_og2)))
13 end
```

Irregular production history (workover dip + choke ripp



```
1 scatter(tsteps_og2, well_data[1, :]; label="reported rate", xlabel="time (yr)",
2   ylabel="q (bbl/d)",
3   title="Irregular production history (workover dip + choke ripple)")
```

2. A neural ODE with no assumed decline law ⇔

The network sees both the current rate and the time, so it can in principle represent time-driven effects like the workover and the periodic ripple, not just state-driven decline.

```
ComponentVector{Float32}(layer_1 = (weight = Float32[0.38878012 1.691749; -0.98136103 0.37034
```

```
1 begin
2   qnet = Chain(Dense(2, 20, tanh), Dense(20, 20, tanh), Dense(20, 1))
3   qnet_ps, qnet_st = Lux.setup(rng_og2, qnet)
4   qnet_ps = ComponentArray(qnet_ps)
5 end
```

```
neural_well_dynamics (generic function with 1 method)
```

```
1 function neural_well_dynamics(u, p, t)
2   input = [u[1] / 1000.0, t] # rescale the rate for a well-conditioned network input
3   ŷ, _ = qnet(input, p, qnet_st)
4   return 1000.0 .* ŷ
5 end
```

```
neural_well_prob = ODEProblem with uType Vector{Float64} and tType Float64. In-place: false
Non-trivial mass matrix: false
timespan: (0.0, 3.0)
u0: 1-element Vector{Float64}:
 1000.0
```

```
1 neural_well_prob = ODEProblem(neural_well_dynamics, u0_og2, tspan_og2)
```

3. Train against the noisy history ↻

```
predict_well (generic function with 1 method)
```

```
1 function predict_well(p)
2   solve(neural_well_prob, Tsit5(); p=p, saveat=tsteps_og2,
3         sensealg=InterpolatingAdjoint(; autojacvec=ZygoteVJP())) |> Array
4 end
```

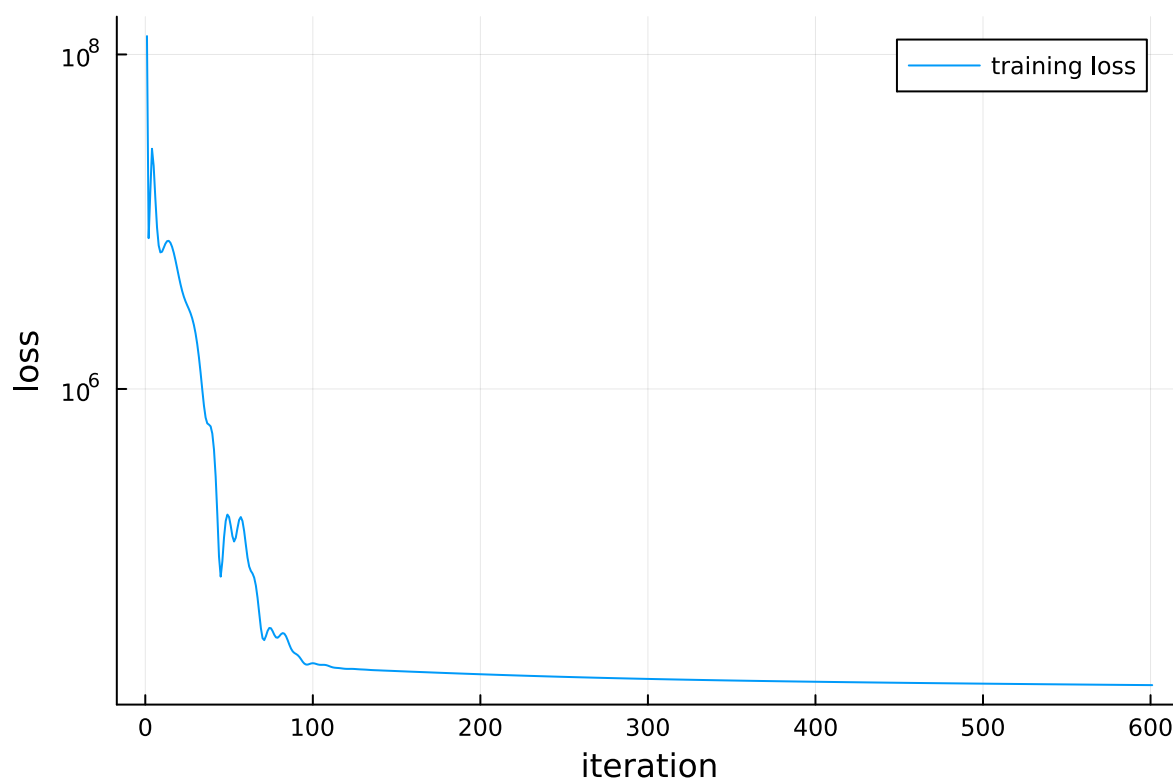
```
loss_well (generic function with 1 method)
```

```
1 function loss_well(p, _)
2   pred = predict_well(p)
3   size(pred) == size(well_data) || return Inf
4   return sum(abs2, well_data .- pred)
5 end
```

► SciMLBase.OptimizationSolution{Float32, 1, ComponentArrays.ComponentVector{Float32, Vector{Float32}}

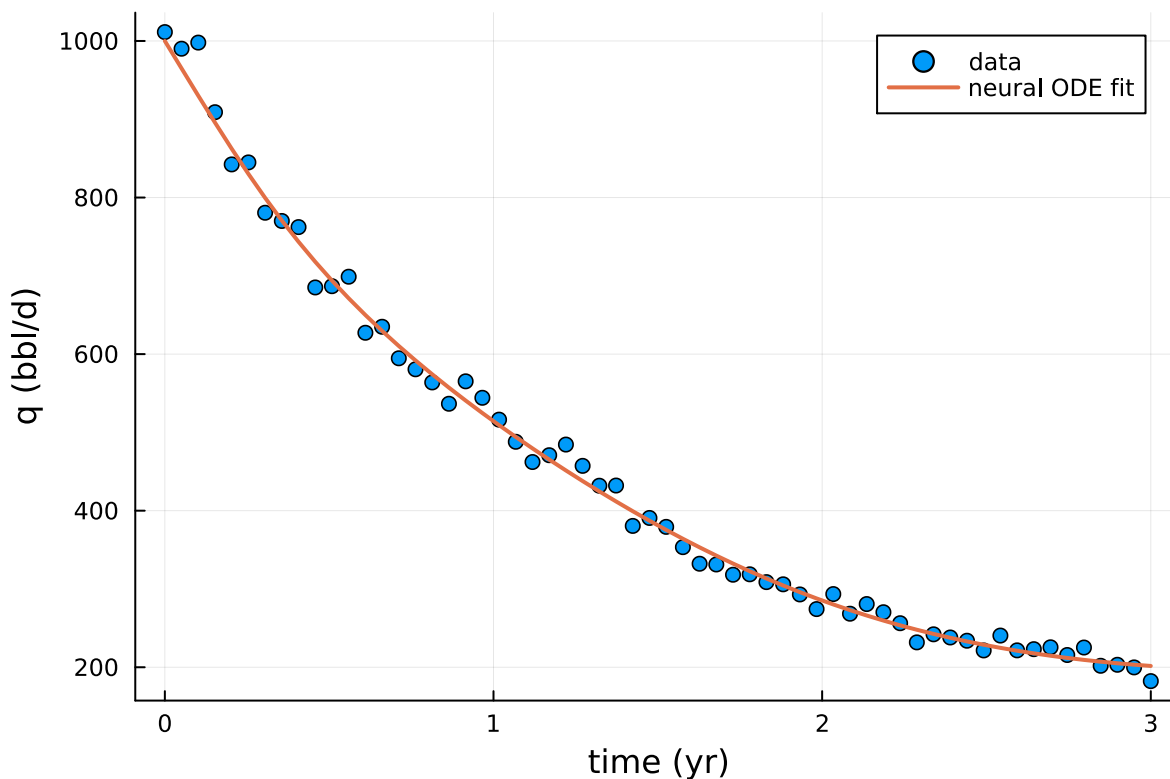
```
1 begin
2     optf_og2 = OptimizationFunction(loss_well, Optimization.AutoZygote())
3     optprob_og2 = OptimizationProblem(optf_og2, qnet_ps)
4
5     well_losses = Float64[]
6     well_callback = function (state, loss)
7         push!(well_losses, loss)
8         return false
9     end
10
11     res_well = Optimization.solve(optprob_og2, OptimizationOptimisers.Adam(0.02);
12     callback=well_callback, maxiters=600)
13 end
```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] x B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.



```
1 plot(well_losses; xlabel="iteration", ylabel="loss", yscale=:log10, label="training loss")
```

4. Does the neural ODE capture the workover and the ripple? ⇌



```
1 begin
2     neural_fit = predict_well(res_well.u)
3     scatter(tsteps_og2, well_data[1, :]; label="data", xlabel="time (yr)", ylabel="q
4             (bbl/d)")
5     plot!(tsteps_og2, neural_fit[1, :]; label="neural ODE fit", lw=2)
6 end
```

⚠ Mixed-Precision `matmul\_cpu\_fallback!` detected and Octavian.jl cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.ReshapedArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange{Int64}}, true}, Tuple{}}] x B [Matrix{Float64}]). Falling back to generic implementation. This may be slow.

Takeaways ⇌

- When the true production mechanism includes operational disturbances that don't belong in any textbook decline equation, a neural ODE fits them anyway — at the cost of a fully black-box model with no physical parameters to report.
- Feeding both q and t into the network lets it represent explicitly time-driven effects (the workover, the periodic ripple), not just state feedback.
- This flexibility-versus-interpretability trade-off motivates the next notebook: keep the *known* physics as an equation, and let a neural network absorb only the part you can't otherwise model.

